পর্ব ৭ · ক্লাস ৩১

কোড বড় হয়, দায়িত্বও

কোডের মানচিত্র · ঋণ কোথায় লেখা · নিরাপদ রিফ্যাক্টর · ডেপেন্ডেন্সি · ডেটা বৃদ্ধি

IMS · ক্লাস ৩১ · স্লাইড ১–১৭

শুরু

এখন কোডটি কত বড়?

আজ গোনা সংখ্যা — অনুমান নয়

অ্যাপ কোড ৫,৪৮০ লাইন

১১টি অ্যাপে, মাইগ্রেশন ও টেস্ট বাদ দিয়ে। সবচেয়ে বড়: `billing` (১,৩৩০)।

টেস্ট কোড ১,৩৫৩ লাইন

অনুপাত ১ : ০.২৫ — অ্যাপ কোডের এক-চতুর্থাংশ। বেশি নয়, কমও নয়।

মোট ৬,৮৩৩ লাইন

৩১ মডেল · ১২ মাইগ্রেশন · ১০৩ টেস্ট · ১০ middleware · ১৯ কমিট

সংখ্যা নিজে থেকে খারাপ নয় — কিন্তু **বড় ফাইল আর নকল যুক্তি** একদিনে তৈরি হয় না, ধীরে জমে। এই ক্লাসের কাজ সেই জমা চেনা।

শিক্ষকের নোট: ক্লাসের শুরুতেই একটি সত্যি কথা: "৬,৮৩৩ লাইন আপনার হাতে লেখা"। এরপর জিজ্ঞেস করুন — "এতে নতুন কেউ ঢুকলে কোথা থেকে শুরু করবে?"

মানচিত্র

১১টি অ্যাপ, ১১টি দায়িত্ব

কোড আর টেস্টের অনুপাতই বলে দেয় কোথায় ঝুঁকি

অ্যাপ	কোড	টেস্ট	অনুপাত	মন্তব্য
billing	১,৩৩০	৪৩৬	১:০.৩৩	সবচেয়ে বড় ও সবচেয়ে পরীক্ষিত
core	৬৫৮	২৪৯	১:০.৩৮	স্কোপিং, পারমিশন, সিড
academics	৫৮৯	৫২	১:০.০৯	**সবচেয়ে কম** — ফাঁক
assessments	৫৬৯	১৫৫	১:০.২৭	নম্বর ও পাবলিশ গেট
lessons	৫২৭	৭৬	১:০.১৪	হোমওয়ার্ক, সিলেবাস
workspaces	৩৯৫	১১৮	১:০.৩০	ব্র্যান্ডিং, ভূমিকা
accounts	৩৬২	১১৪	১:০.৩১	লগইন, রিসেট
students	৩৫৫	৬৮	১:০.১৯	ভর্তি ও স্থানান্তর
attendance	৩৪৩	৮৫	১:০.২৫	নীতি ও সংশোধন
reports · audit	৩৫২	০	—	**শূন্য টেস্ট** — ক্লাস ২৬-এ চিহ্নিত

শিক্ষকের নোট: ছকে দুইটি লাল দাগ দেখান — academics-এর কম অনুপাত আর reports/audit-এর শূন্য। "কোড বড় হওয়ার সঙ্গে সঙ্গে টেস্টও বাড়ে না — সেটাই ঋণ।"

বড় ফাইল

পাঁচটি ফাইল, পাঁচটি সংকেত

ফাইলের আকার নিজে দোষ নয় — কিন্তু সীমা দরকার

ফাইল	লাইন	সংকেত
`apps/billing/tests.py`	৪৩৬	টেস্ট বেশি = ভালো; তবে এক ফাইলে সব = খুঁজতে সময়
`apps/billing/views_platform.py`	৩১৭	এক ফাইলে বহু ভিউ — ভাগ করার প্রার্থী
`core/management/commands/seed_demo.py`	২৮১	ডেমো ডেটার বর্ণনা — লম্বা হওয়াই স্বাভাবিক
`apps/billing/models.py`	২৪৯	Plan + Subscription + Invoice + Payment — তিন মডেল এক ফাইলে
`apps/lessons/views.py`	২৪৮	৯ ভিউ — স্কোপিং নিয়ম একই থাকলে ভাগ করা সহজ

নিয়ম: **৩০০ লাইনের বেশি = থামুন ও ভাবুন**। কিন্তু ভাগ করবেন তখনই, যখন টেস্ট আগে থেকে আছে — নইলে ভাগ করা মানে ঝুঁকি নেওয়া।

শিক্ষকের নোট: জিজ্ঞেস করুন: "views_platform.py ভাগ করলে কী লাভ, কী ক্ষতি?" লাভ: পড়া সহজ; ক্ষতি: ইমপোর্ট আর URL নাম বদলাতে হবে।

ঋণের খাতা

ঝুঁকি কোথায় লেখা থাকে

IMS-এ কোডে একটিও `TODO` নেই

কোডে মন্তব্য-ঋণ

`# TODO: পরে ঠিক করব` — গোনা হয়েছে **০**। ভালো লক্ষণ, তবে মানে এই নয় ঋণ নেই।

যেখানে আসলে লেখা

`BUILD_STATUS.md` — "Remaining work (not faked, not skipped)" · `DEPLOYMENT.md` — "not yet deployed"

কেন এভাবে

একটি তালিকা সবার চোখে থাকে; ছড়িয়ে থাকা `TODO` কেউ পড়ে না।

ঝুঁকি

খাতা হালনাগাদ না হলে ঋণ আবার লুকিয়ে যায় — তাই রিলিজের সঙ্গে খাতা বদলাতে হবে।

```
## Remaining work
1. Invitation workflow
2. Platform console
4. Backup & restore console
5. Security attack suite

## Verified numbers
test            ->  90 tests, 90 OK
smoke_test.py   ->  55 pages x 4 roles
check --deploy  ->  no issues
```

শিক্ষকের নোট: কৌশলটি শেখান: ঋণ লিখুন **এক জায়গায়**, আর প্রতিটি সংখ্যার সঙ্গে প্রমাণের কমান্ড। এটাই ক্লাস ২৭-এর প্রমাণ-খাতার ধারাবাহিকতা।

সিদ্ধান্ত

কখন রিফ্যাক্টর করবেন, কখন নয়

"তিনবার নিয়ম" — শেখার সবচেয়ে কাজের নিয়ম

তিনবার নিয়ম

একই জিনিস প্রথমবার লিখলে — শুধু লিখুন।
দ্বিতীয়বার — মেনে নিন।
তৃতীয়বার — তখনই সাধারণ রূপ বের করুন।

করুন	করবেন না
তিনবার একই যুক্তি	ভবিষ্যতে "হতে পারে" বলে
বাগের পরে, টেস্ট লেখার পর	টেস্ট ছাড়া
একটি ছোট ধাপে	পুরো ফাইল একসাথে
কাজের অংশ হিসেবে	"শুধু রিফ্যাক্টরের" আলাদা সপ্তাহে

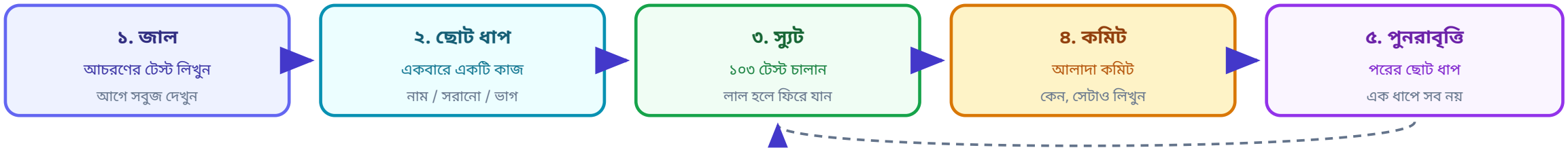
IMS-এ বাস্তব: `BootstrapFormMixin` আর `ImsFormMixin` — দুটি mixin, একই কাজ। তৃতীয়বার দরকার হলেই এক করা উচিত।

শিক্ষকের নোট: "Rule of three" মুখস্থ করান। শিক্ষার্থীরা প্রায়ই আগেভাগে abstraction বানায় — সেটিই সবচেয়ে বেশি ক্ষতি করে।

প্রোটোকল

নিরাপদ রিফ্যাক্টরের পাঁচ ধাপ

প্রতিটি ধাপের শেষে স্যুট সবুজ — নইলে পিছিয়ে যান



একমাত্র অনুমোদন: রিফ্যাক্টরে কোনো টেস্ট **বদলানো যাবে না**। টেস্ট বদলাতে হলে আচরণ বদলেছে — সেটি নতুন ফিচার, রিফ্যাক্টর নয়।

বাস্তব সময়: ১০০ লাইনের রিফ্যাক্টর ২০ মিনিট; ১,০০০ লাইনের "রিফ্যাক্টর" দুই দিন — আর সেটি প্রায়ই ব্যর্থ হয়।

শিক্ষকের নোট: বোর্ডে ৫ ধাপ লিখে রাখুন; ক্লাসের বাকি অংশে প্রতিটি উদাহরণে এই ধাপগুলোই দেখাবেন।

নকল

দুই mixin, এক কাজ

কীভাবে এক করবেন — টেস্ট আগে

```
# জাল: আচরণ লিখে ফেলুন (কোড বদলানোর আগে)
def test_every_form_field_gets_design_classes(self):
    f = StudentForm(workspace=self.ws).fields
    self.assertIn("form-control", f["full_name"].widget.attrs["class"])
    self.assertIn("form-select", f["class_group"].widget.attrs["class"])

# ধাপ ১: সবুজ দেখুন (দুই কপি এখনো আছে)
# ধাপ ২: একটিকে apps/core/forms.py-তে নিন
#   class DesignClassesMixin:  ...  (এক জায়গায়)
# ধাপ ৩: academics আর billing দুটোই
#         from apps.core.forms import DesignClassesMixin
# ধাপ ৪: পুরো স্যুট — ১০৩ টেস্ট সবুজ = আচরণ অপরিবর্তিত
```

কেন দুই কপি বিপজ্জনক

এক কপিতে বাগ ঠিক করলে অন্য কপিতে থাকে — "কিছু ফর্মে ত্রুটি দেখায় না" ধরনের অদ্ভুত বাগ।

ধাপ ৩-এর ফাঁদ

একসাথে দুই ফাইল বদলালে কোনটি ভাঙল বোঝা যায় না — ফাইল ধরে ধরে এগোন, প্রতি ধাপে স্যুট।

নতুন জায়গা

`apps/core/forms.py` — শেয়ারড ফর্ম-আচরণের ঘর; অ্যাপগুলো এখান থেকে নেয়।

শিক্ষকের নোট: জোর দিন: নকল সরানোর আগে **টেস্ট** — নইলে "সব ঠিক আছে" বলার প্রমাণ থাকে না।

নির্ভরতা

৯টি প্যাকেজ, সবগুলোর নির্দিষ্ট সংস্করণ

ওপেন রেঞ্জ নয় — পিন করা

```
# requirements.txt (dev + app)
Django==6.1.2
psycopg[binary]==3.3.6
python-dotenv==1.2.4
Pillow==12.3.0
openpyxl==3.1.5      # এক্সেল এক্সপোর্ট
reportlab==5.1.0     # মার্কশিট PDF

# requirements-prod.txt
-r requirements.txt
gunicorn==23.0.0
whitenoise==6.11.0
dj-database-url==3.0.1
```

কেন `==`

আজ যা পরীক্ষা করেছেন, কাল সার্ভারে ঠিক সেটাই বসবে — নীরব আপডেট নেই।

কীভাবে আপডেট

একটি প্যাকেজ → শাখা → `pip install -U` → টেস্ট → স্টেজিং → ডিপ্লয়।

নিরাপত্তা আপডেট

Django-র নিরাপত্তা রিলিজ বাদ দেওয়া সবচেয়ে ব্যয়বহুল অবহেলা — মাসে একবার দেখার অভ্যাস।

ফাঁক

অটোমেটেড নির্ভরতা-স্ক্যান (dependabot/audit) নেই — হাতে দেখতে হয়।

শিক্ষকের নোট: প্রশ্ন: "উপকারী প্যাকেজ যোগ করার আগে কী ভাববেন?" উত্তর: রক্ষণাবেক্ষণ কে করে, লাইসেন্স কী, আর সরানোর খরচ কত।

আপগ্রেড

ফ্রেমওয়ার্ক আপডেট — ভয় নয়, ক্রম

Django 6.1.2-এ দাঁড়িয়ে আছে; পরের লাফের প্রস্তুতি

#	ধাপ	কেন
১	রিলিজ নোট পড়ুন (deprecations)	কী সরছে, তা আগেই জানা
২	ওয়ার্নিং চালু করে টেস্ট	`python -W error::DeprecationWarning manage.py test`
৩	এক মাইনর সংস্করণ করে এগোন	৬.১ → ৬.২, একসাথে ৬.১ → ৭.০ নয়
৪	পূর্ণ স্যুট + `smoke_test` + `check --deploy`	তিনটি ভিন্ন দৃষ্টি
৫	প্রথমে স্টেজিং/রিহার্সাল, পরে আসল	`deploy_check.sh` এখানেই কাজে লাগে

IMS-এর সুবিধা: `override_settings` আর `django.conf.settings` ব্যবহার — হার্ডকোড কম, তাই আপগ্রেডে কম ভাঙে।

ফাঁদ: আপগ্রেড আর নতুন ফিচার একই রিলিজে নয় — ভাঙলে কারণ দুটির কোনটি, বোঝা যায় না।

শিক্ষকের নোট: শিক্ষার্থীদের বলুন: "আপগ্রেড একটি কাজ, ফিচার আরেকটি কাজ" — একসাথে করলেই বিভ্রান্তি।

বৃদ্ধি

ডেটা বাড়লে কী বাড়ে

ডেমোর সংখ্যা বনাম এক বছরের বাস্তবতা

টেবিল	ডেমোতে আজ	৮০০ ছাত্র · ১ বছর
ছাত্র প্রোফাইল	১০	৮০০
উপস্থিতি সারি	২০০	৮০০ × ২০০ দিন ≈ **১,৬০,০০০**
নম্বর	৩০	৮০০ × ৮ বিষয় × ৩ পরীক্ষা ≈ **১৯,২০০**
হোমওয়ার্ক জমা	১৫	৮০০ × ৩০ ≈ ২৪,০০০
অডিট সারি	০	প্রতিদিন ১০০ কর্ম ≈ **৩৬,৫০০**

মূল কথা: ২ লাখ সারিতে ইনডেক্স ছাড়া তালিকা খুললে সেকেন্ডের হিসাব বদলে যায়।

IMS-এ আছে: ১৪টি `db_index` / `indexes` ঘোষণা আর ৫০টি unique সীমা — তবে সব কোয়েরি মাপা হয়নি।

শিক্ষকের নোট: বোর্ডে হিসাবটি করুন: ৮০০ × ২০০ = ১,৬০,০০০। শিক্ষার্থীরা নিজে গুণ করলে সংখ্যাটি মনে থাকে।

ধীর কোয়েরি

N+1 — সবচেয়ে সাধারণ ধীরতা

লুপের ভেতরে কোয়েরি = বিপদ

```
# খারাপ: প্রতি সারিতে একটি কোয়েরি (N+1)
for st in students:            # ৮০০ কোয়েরি
    print(st.class_group.name) # +৮০০

# ভালো: এক কোয়েরিতে সব
students = StudentProfile.objects.filter(
    workspace=ws).select_related("class_group")   # ১ কোয়েরি

# অনেক-এর-অনেক হলে
    .prefetch_related("enrollments__class_group")

# মাপার উপায় (dev-এ)
from django.test.utils import CaptureQueriesContext
from django.db import connection
with CaptureQueriesContext(connection) as ctx:
    list(students)
print(len(ctx.captured_queries))     # সংখ্যাটাই প্রমাণ
```

কেন চোখে ধরা পড়ে না

ডেমোতে ১০ ছাত্র — ৫ মিলিসেকেন্ড। বাস্তবে ৮০০ — ৪ সেকেন্ড।

মাপার অভ্যাস

নতুন তালিকা-ভিউ লিখলে একবার কোয়েরি গুনে নিন; সংখ্যা জানলে উন্নতি বোঝা যায়।

ক্যাশ আগে নয়

ভুল কোয়েরি ক্যাশ করলে ভুল উত্তর দ্রুত আসে — আগে কোয়েরি ঠিক করুন।

শিক্ষকের নোট: `CaptureQueriesContext` ক্লাসে চালিয়ে দেখান — সংখ্যা মুদ্রিত হলে "অনুভূতি" বদলে যায়।

মাইগ্রেশন

১২টি মাইগ্রেশন — কখন জমানো থামাবেন

ইতিহাস মুছবেন না, তবে গুছিয়ে রাখুন

১১ অ্যাপে ১২টি মাইগ্রেশন

প্রতিটি অ্যাপের প্রথম মাইগ্রেশনেই বেশিরভাগ মডেল — ছোট, পড়ার মতো ইতিহাস।

squash কখন

একই অ্যাপে ২০+ মাইগ্রেশন হলে, বা পুরোনো রিলিজ আর দরকার না হলে — `squashmigrations`।

সবচেয়ে বিপজ্জনক

ইতিমধ্যে ডিপ্লয় করা মাইগ্রেশন সম্পাদনা করা — অন্য পরিবেশে ইতিহাস অমিল হয়ে যায়।

যাচাই

`makemigrations --check --dry-run` — মডেল আর মাইগ্রেশন অমিল হলে ধরা পড়ে।

```
# নিয়মিত তিনটি কমান্ড
python manage.py makemigrations --check --dry-run
python manage.py showmigrations
python manage.py migrate --plan

# শাখা মিলিয়ে নেওয়ার পর
python manage.py migrate --merge
```

শিক্ষকের নোট: বলুন: "মাইগ্রেশন ফাইল হলো ডেটাবেসের ইতিহাস — ইতিহাস নতুন করে লেখা যায় না, শুধু সামনে এগোয়।"

হাতবদল

নতুন ডেভেলপারের প্রথম দিন

দস্তাবেজ যদি কাজ করে, তবে কোড পড়া কম লাগে

#	কাজ
১	`START_HERE.md` — দুই পথ: SQLite দ্রুত, PostgreSQL আসল
২	`bash run.sh` → চারটি লগইন দিয়ে ঢুকে দেখুন
৩	`BUILD_STATUS.md` — কী তৈরি, কী বাকি, কোন প্রমাণে
৪	`python manage.py test` → ১০৩ সবুজ দেখুন
৫	একটি ছোট বাগ ঠিক করে টেস্টসহ কমিট

প্রতিটি ধাপের একটি **যাচাইযোগ্য ফল** আছে — "পড়ে নিন" নয়, "চালিয়ে দেখুন"।

দস্তাবেজ যা আছে

START_HERE · README · BUILD_STATUS · DEPLOYMENT · LAUNCH_PLAYBOOK · USER_MANUAL (বাংলা) · ৫৩ স্ক্রিনশট

যা নেই

আর্কিটেকচার সিদ্ধান্তের আলাদা নথি (ADR) — কেন এভাবে, তা কোড-মন্তব্যেই ছড়িয়ে আছে।

শিক্ষকের নোট: ছাত্রদের বলুন: "আপনার প্রজেক্টে নতুন কেউ এলে প্রথম দিনের পাঁচ ধাপ লিখে দিন" — এটাই হাতবদলের প্রস্তুতি।

অনুশীলন ও আলোচনা

২০ মিনিট — নিজের কোডের ঋণ মাপুন

চারটি কাজ, সবই মাপা যায়

১. বড় ফাইল খুঁজুন

```
find . -name "*.py" | xargs wc -l | sort -rn | head
```

২. কোয়েরি গুনুন

একটি তালিকা-ভিউতে `CaptureQueriesContext` দিয়ে সংখ্যা লিখে রাখুন।

৩. নকল খুঁজুন

একই ৫ লাইনের বেশি দুই জায়গায়? তালিকায় লিখুন।

৪. ঋণ-খাতা লিখুন

তিনটি কাজ, প্রতিটির সঙ্গে কারণ ও ঝুঁকি।

- বড় ফাইল ভাঙলে কী লাভ?
- N+1 ধরা পড়ে কীভাবে?
- Django আপগ্রেড কেন ধাপে ধাপে?
- প্যাকেজ পিন না করলে কী হয়?
- ৮০০ ছাত্রে কোন টেবিল সবচেয়ে বড়?

শিক্ষকের নোট: শেষ প্রশ্নের উত্তর: উপস্থিতি (attendance) — ১,৬০,০০০ সারি। এটাই প্রথম জায়গা যেখানে ইনডেক্স দরকার হবে।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

রক্ষণাবেক্ষণের প্রথম ক্লাসের সারাংশ

নীতি	প্রমাণ
কোড বাড়লে দায়িত্ব বাড়ে	৫,৪৮০ লাইন · ৬,৮৩৩ মোট
টেস্টের অনুপাতই ঝুঁকির মানচিত্র	academics ১:০.০৯ · reports/audit ০
ঋণ এক জায়গায় লিখুন	`BUILD_STATUS.md` · ০ `TODO`
তিনবার নিয়ম	দুই mixin — তৃতীয়বারে এক করুন
রিফ্যাক্টরের অনুমোদন = সবুজ স্যুট	১০৩ টেস্ট অপরিবর্তিত
বৃদ্ধি মাপুন, অনুমান নয়	১,৬০,০০০ উপস্থিতি সারি/বছর

পরের ক্লাসে: এই কোড দল বদলালে, দায়িত্ব হাতবদল হলে, আর এক বছর পরের IMS কেমন হবে।

শিক্ষকের নোট: শিক্ষার্থীদের বলুন নিজের প্রজেক্টের জন্য এই ছয়টি সারির একটি নিজের ভাষায় লিখতে।

০২

এক বছর পরের IMS

হাতবদল ও দল · সিদ্ধান্তের নথি · রোডম্যাপ · ফিচার বনাম ঋণ · কখন থামবেন

পর্ব ৭ · ক্লাস ৩২

এক বছর পরের IMS

রোডম্যাপ · সিদ্ধান্তের নথি · দল ও রিলিজ · ফিচার বনাম ঋণ · কখন নতুন করে লিখবেন না

IMS · ক্লাস ৩২ · স্লাইড ১৮–৩৪

শুরু

৩৬৫ দিনে চারটি জিনিস বদলায়

আর প্রতিটির জন্য আলাদা প্রস্তুতি দরকার

কী বদলায়	কেমন দেখায়	প্রস্তুতি
ব্যবহারকারী	৪ জন → ৪০ জন; কেউ প্রশিক্ষণ নিয়ে আসে, কেউ নয়	ব্যবহারবিধি ও সহায়তা (USER_MANUAL)
ডেটা	২০০ উপস্থিতি → ১,৬০,০০০ সারি	ইনডেক্স, ব্যাকআপ, আর্কাইভ
কোড	৫,৪৮০ → সম্ভবত ৮,০০০ লাইন	টেস্টের অনুপাত ধরে রাখা
মানুষ	আপনি একা → একজন সহকারী, বা কেউ নেই	দস্তাবেজ ও সিদ্ধান্তের নথি

চারটির মধ্যে **শুধু কোড** নিজে থেকে ঠিক থাকে না বাকি তিনটি পরিকল্পনা ছাড়া এলোমেলো হয় — আর সেটাই এক বছর পরের আসল ঝুঁকি

শিক্ষকের নোট: জিজ্ঞেস করুন: "এক বছর পর আপনার প্রজেক্টে সবচেয়ে বড় সমস্যা কী হবে?" উত্তর প্রায় সবসময় ডেটা বা মানুষ, কোড নয়

রোডম্যাপ

১২ মাসের তিনটি ধাপ

প্রতিটি ধাপের সঙ্গে একটি যাচাইযোগ্য লক্ষ্য

সময়	কাজ	লক্ষ্য / প্রমাণ
মাস ১–৩	মূল ব্যবহার শক্ত করা: মার্কস এন্ট্রি, উপস্থিতি, রিপোর্ট কার্ড	প্রতিদিন ব্যবহার, ৫০০ = ০
মাস ১–৩	ইনভাইট ও CSV আমদানি (অনবোর্ডিংয়ের ধীর ধাপ)	একটি স্কুল ২০ মিনিটে ওঠে
মাস ৪–৬	মনিটরিং, CI, রেট লিমিট, ব্যাকআপ কনসোল	অ্যালার্ট আসে, রিস্টোর ড্রিল পাস
মাস ৪–৬	academics-এর টেস্ট (১:০.০৯ → ১:০.৩)	টেস্ট সংখ্যা বাড়ে, বাগ কমে
মাস ৭–১২	নতুন ফিচার (শিক্ষক-উপস্থিতি, এসএমএস, পেমেন্ট গেটওয়ে)	গ্রাহক-অনুরোধের তালিকা থেকে
সারা বছর	প্রতি রিলিজে দস্তাবেজ হালনাগাদ	BUILD_STATUS সংখ্যা সত্যি

নিয়ম: রোডম্যাপে **তারিখের চেয়ে ক্রম বেশি গুরুত্বপূর্ণ** — প্রথম তিন মাসের কাজ শেষ না করে চতুর্থ মাসের ফিচারে যাবেন না

শিক্ষকের নোট: BUILD_STATUS-এর "Remaining work" তালিকাটি এই রোডম্যাপের বীজ — ক্লাসে দুটিকে পাশাপাশি রেখে দেখান

ADR

সিদ্ধান্তের নথি — কেন এভাবে

কোড বলে "কী", ADR বলে "কেন"

```
# ADR-004: সাবস্ক্রিপশন গেট middleware-এ

## প্রসঙ্গ
মেয়াদ শেষ হলে কিছুই লেখা যাবে না, কিন্তু পড়া
আর এক্সপোর্ট চালু থাকবে ২০+ ভিউ

## সিদ্ধান্ত
প্রতি ভিউয়ে নয় — এক middleware-এ
SAFE_METHODS বাদ, ALLOW_PREFIXES ছাড়

## বিকল্প যা বাদ গেল
ভিউ ডেকোরেটর: ২০ জায়গায় একই কোড, একটায়
ভুল হলে ফাঁক

## পরিণতি
নতুন ভিউ স্বয়ংক্রিয়ভাবে সুরক্ষিত; কিন্তু
বুক হলে ডিবাগ করা কঠিনতর
```

IMS-এর চারটি আসল সিদ্ধান্ত

modular monolith (মাইক্রোসার্ভিস নয়) · কাস্টম User (email লগইন) · বিলিং অ্যাপের ভেতরে · AI/বাহ্যিক সেবা নেই

গঠন

প্রসঙ্গ → সিদ্ধান্ত → বাদ পড়া বিকল্প → পরিণতি চারটি শিরোনাম, এক পাতা

এখনো নেই

IMS-এ আলাদা `docs/adr/` ফোল্ডার নেই — চার নম্বর তথ্যটি কোড-মন্তব্য ও ক্লাসে ছড়িয়ে আছে

শিক্ষকের নোট: বলুন: "ছয় মাস পর কেউ জিজ্ঞেস করবে 'কেন middleware, ভিউ ডেকোরেটর নয়?' — ADR সেই উত্তর" ছাত্রদের একটি ADR লিখিয়ে নিন

কোথায় কী লেখে

চার জায়গা, চার দায়িত্ব

একই তথ্য দুই জায়গায় লিখলে একদিন অমিল হবে

জায়গা	কী থাকে	কে পড়ে
কমিট বার্তা	এই পরিবর্তনটি কী ও কেন	কোড-ইতিহাস খুঁড়তে যাওয়া কেউ
ADR	বড় সিদ্ধান্ত ও বাদ পড়া বিকল্প	নতুন টিম-সদস্য
`BUILD_STATUS.md`	কী তৈরি, কী বাকি, কোন প্রমাণে	গ্রাহক, আপনি নিজে, নতুন ডেভেলপার
ইনলাইন মন্তব্য	অদ্ভুত কোডের **কারণ** (কী নয়)	পাশের লাইন পড়তে যাওয়া কেউ

ভালো মন্তব্য: "সেশন ওয়ার্কস্পেস আর URL আলাদা হতে পারে — তাই slug থেকে পড়ি" (কেন)

খারাপ মন্তব্য: "ওয়ার্কস্পেস নিন" — কোডেই তো দেখা যাচ্ছে মন্তব্য যা বলে তা নয়, যা বলে না তা লিখুন

শিক্ষকের নোট: এই স্লাইডটি প্রায়ই শিক্ষার্থীদের ধারণা বদলায়: "কোড নিজেই দস্তাবেজ" — আধা-সত্য; কোড "কী" বলে, "কেন" নয়

দল

একজন থেকে তিনজন

ভূমিকা আলাদা করুন, নিয়ম লিখিত রাখুন

ভূমিকা	দায়িত্ব
রক্ষক (maintainer)	মার্জ, রিলিজ, মাইগ্রেশন — একজনের হাতে
নির্মাতা	ফিচার + টেস্ট একসাথে
সহায়তা (support)	টিকেট, ব্যবহারকারীর প্রশ্ন, ডেটা সংশোধন
গ্রাহক-সফলতা	অনবোর্ডিং, বিলিং, প্রশিক্ষণ

তিনটি অলঙ্ঘনীয় নিয়ম

১. টেস্ট ছাড়া মার্জ নয়
২. মাইগ্রেশন রিভিউ ছাড়া ডিপ্লয় নয়
৩. রিলিজের আগে ব্যাকআপ

বাস্তব ঝুঁকি

সব সিদ্ধান্ত একজনের মাথায় = **bus factor ১**; ছুটি বা অসুস্থতাতেই প্রকল্প থেমে যায়

দল ছোট হলে নিয়ম আরও লিখিত হতে হয় — মুখে বলা নিয়ম কেবল উপস্থিত থাকা মানুষদের কাজে লাগে

শিক্ষকের নোট: ক্লাসে "bus factor" ধারণাটি ব্যাখ্যা করুন: "আপনি অসুস্থ হলে কত দিন প্রকল্প চলবে?"

রিলিজ

সংস্করণ, ট্যাগ ও পরিবর্তন-তালিকা

যাতে "কোন সংস্করণে কী ছিল" প্রশ্নের উত্তর থাকে

```
# সংস্করণ: MAJOR.MINOR.PATCH
1.4.0   নতুন ফিচার (ইনভাইট)
1.4.1   বাগ ঠিক
2.0.0   ভাঙা পরিবর্তন (URL বদল)

# ট্যাগ ও নোট
git tag -a v1.4.0 -m "Invitations + CSV import"
git push --tags

CHANGELOG.md
## 1.4.0 — 2026-12-01
+ invitation workflow (F-10..12)
* academics tests 52 -> 90
! migration: students 0004
```

কেন MAJOR লাগে

গ্রাহক বলবেন "আপনাদের ২.০-তে URL বদলেছে" — তখন উত্তর প্রস্তুত

মাইগ্রেশন উল্লেখ করুন

রিলিজ নোটে মাইগ্রেশন না লিখলে কেউ ব্যাকআপ নেয় না

আজকের বাস্তব

IMS-এ ১৯টি কমিট আছে, কিন্তু কোনো git tag বা CHANGELOG নেই — যোগ করা প্রথম কাজগুলোর একটি

শিক্ষকের নোট: শিক্ষার্থীদের বলুন প্রতিটি রিলিজে তিনটি জিনিস লিখতে: তারিখ, কী বদলাল, কোন মাইগ্রেশন

বাজেট

ফিচার বনাম ঋণ — ৭০ / ২০ / ১০

প্রতি রিলিজের কাজ ভাগ করে নিন

৭০% — নতুন ফিচার (গ্রাহক যা চায়) | **২০% ঋণ** | **১০%**

ফিচার — ব্যবহারকারীর কাজ সহজ করে

ঋণ — টেস্ট, রিফ্যাক্টর, ইনডেক্স, দস্তাবেজ

১০% — অনিশ্চিত কাজ / পরীক্ষা (নতুন ফিচার বা ঝুঁকি)

ঋণ শূন্য হলে ঋণ ৩০% করুন

কখনো ০% নয়

IMS-এর ঋণ-তালিকা বাস্তবে: academics টেস্ট · reports/audit টেস্ট · CI · healthz · রেট লিমিট · ৫০০ পাতার reference id

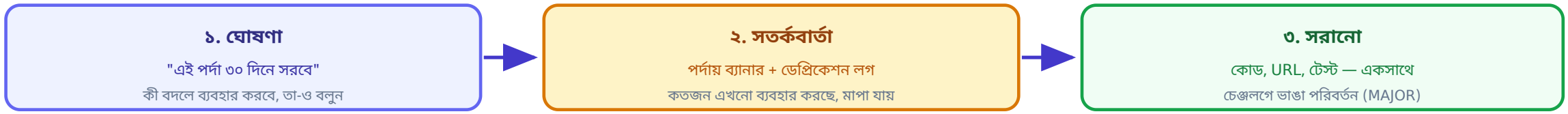
ফাঁদ: ঋণ ০% রাখলে কয়েক মাস পর ফিচারের গতি অর্ধেক — প্রতিটি ফিচার আগের ফিচারের কাঁটায় আটকে যায়

শিক্ষকের নোট: "ঋণ শোধ করা মানে নতুন কাজ না করা" — এই ভুল ধারণাটি ভাঙুন; ২০% নিয়মিত রাখলে বড় সংকট আসে না

ডেপ্রিকেশন

ফিচার বন্ধ করা — তিন ধাপ

হঠাৎ বন্ধ করলেই ব্যবহারকারী হারাবেন



উদাহরণ যেভাবে হবে: ধরা যাক ম্যানুয়াল bKash এন্ট্রি বদলে গেটওয়ে আসছে — পুরোনো পর্দা ৩০ দিন ব্যানারসহ থাকবে, তারপর সরবে ডেটা (`Invoice.reference`) থাকবে

কখনো করবেন না: ডেটা সরানো পর্দা সরানো যায়, ইতিহাস নয় — অডিট আর ইনভয়েস চিরকাল থাকা দরকার

শিক্ষকের নোট: "কোড মুছে ফেলা সহজ, ব্যবহারকারীর অভ্যাস মুছে ফেলা কঠিন" — তিন ধাপের কারণ এটাই

গোপনীয়তা

কোন ডেটা কত দিন রাখবেন

অডিট, ছাত্র, ব্যাকআপ — তিনটি আলাদা নিয়ম

ডেটা	কত দিন	কারণ
নম্বর, উপস্থিতি	যত দিন প্রতিষ্ঠান গ্রাহক	রিপোর্ট কার্ড ও বছরের তুলনা দরকার
`AuditLog`	অন্তত ১ বছর	"কে বদলাল" প্রশ্ন বছরের শেষেও আসে
ব্যাকআপ	১৪ দিন (বর্তমানে)	ভুল ধরা ও ফেরানোর জন্য যথেষ্ট
লগ	আয়োজক যত রাখে	ডিবাগিংয়ের জন্য; ব্যক্তিগত তথ্য নয়
মুছে ফেলা প্রতিষ্ঠান	প্রথমে ব্যাকআপ, পরে মুছে ফেলা	UI-তে ডিলিট নেই — ইচ্ছাকৃত

অনুরোধ এলে: "আমার ডেটা দিন" → রিপোর্ট/এক্সপোর্ট (CSV, XLSX, PDF) আছে; "মুছুন" → ব্যাকআপ নিয়ে, মালিকের অনুমোদনে

যা লগে রাখবেন না: পাসওয়ার্ড, রিসেট টোকেন, পুরো ছাত্র-রেকর্ড — `AuditLog`-এর নিয়মেই লেখা

শিক্ষকের নোট: বাংলাদেশে নাগরিক তথ্য সুরক্ষা নিয়ে আলোচনা চলছে; নীতি এখনই লিখে রাখলে ভবিষ্যতে সুবিধা — "কম রাখুন, দরকারে রাখুন"

খরচ

এক বছরে খরচ কত

৮০০ ছাত্র, একটি প্রতিষ্ঠান — মোটামুটি হিসাব

খাত	মাসিক (আনুমানিক)	নোট
অ্যাপ হোস্টিং (ছোট ইনস্ট্যান্স)	মার্কিন $৭–২৫	Render/Railway/Fly ফ্রি টিয়ারও চলে
PostgreSQL (ম্যানেজড)	$০–১৫	ফ্রি টিয়ারে সীমা; ব্যাকআপ আলাদা
ডোমেইন	বছরে ~$১২	কাস্টম ডোমেইন ক্লায়েন্টের
ইমেইল (৩০০০/মাস)	$০–১০	Brevo ফ্রি টিয়ার যথেষ্ট শুরুতে
স্টোরেজ (`media/`)	কয়েক ডলার	লোগো ছোট, পড়ার উপকরণ বড় হতে পারে

আয়: স্টার্টার ৳১,৫০০/মাস — এক প্রতিষ্ঠানই হোস্টিংয়ের কয়েকগুণ কভার করে

লুকানো খরচ: আপনার সময় সাপোর্ট, ব্যাকআপ যাচাই, আপডেট — এগুলোই আসল মাসিক ব্যয়

শিক্ষকের নোট: সংখ্যাগুলো আজকের বাজারদর নয়, মাত্রা বোঝানোর জন্য — শিক্ষার্থীদের নিজে যাচাই করতে বলুন

ফাঁদ

"নতুন করে লিখে ফেলি" — প্রায় সবসময় ভুল

পুরোনো কোডে যা আছে, তা দেখা যায় না — কিন্তু থাকে

পুরোনো কোডে লুকানো আছে	নতুন কোডে
এজ-কেস: অনুপস্থিত ছাত্র, মেয়াদোত্তীর্ণ ট্রায়াল	প্রথম থেকে শিখতে হবে
স্কোপিংয়ের সূক্ষ্ম নিয়ম	আবার বাগ হবে
১০৩ টেস্টের জ্ঞান	শূন্য থেকে শুরু
গ্রাহকের অভ্যস্ত পর্দা	আবার প্রশিক্ষণ
মাইগ্রেশন-ইতিহাস	ডেটা সরানোর ঝুঁকি

বিকল্প: ধীরে বদল (strangler)

একটি অংশ নতুন করে লিখুন, পুরোনোটা পাশে রেখে — ট্রাফিক ধীরে সরান, তারপর পুরোনোটা সরান

IMS-এ কোথায় প্রযোজ্য

রিপোর্ট বা রফতানি ধীর হলে সেটিকে আলাদা সার্ভিস করা যায় — পুরো সিস্টেম নয়

প্রথম প্রশ্ন

"কোন সমস্যার সমাধান করছি?" উত্তর না থাকলে রিওয়াইট নয়, রিফ্যাক্টর করুন

শিক্ষকের নোট: জোর দিন: "রিওয়াইট মানে পুরোনো জ্ঞান ফেলে নতুন করে শেখা" — াসে এটি সবচেয়ে দরকারি সতর্কতা

প্রযুক্তি

ফ্রেমওয়ার্ক বা ভাষা বদলাবেন কখন

প্রায় কখনোই কারিগরি কারণে নয়

যুক্তিসঙ্গত কারণ

ফ্রেমওয়ার্ক আর রক্ষণাবেক্ষণ পাচ্ছে না (নিরাপত্তা আপডেট নেই) · স্কেলিংয়ে প্রকৃত দেয়াল · পুরো দলের দক্ষতা অন্য স্ট্যাকে

অজুহাত

"নতুন প্রযুক্তি বেশি আধুনিক" · "অন্যরা করছে" · "আমার পছন্দের নয়" — এগুলো দিয়ে শুরু হলে বছর শেষে কিছুই তৈরি হয় না

লক-ইন হিসাব

Django, PostgreSQL, WhiteNoise — সবগুলো ওপেন ও প্রতিস্থাপনযোগ্য সবচেয়ে বড় লক-ইন আপনার নিজের কোড, ফ্রেমওয়ার্ক নয়

IMS-এ Django 6.1.2 · Python 3.13 · PostgreSQL — দশ বছরের বেশি রক্ষণাবেক্ষণের ইতিহাস আছে এমন স্ট্যাক; নিরাপত্তা আপডেটই আসল কারণ

শিক্ষকের নোট: প্রশ্ন: "নতুন ফ্রেমওয়ার্কে যেতে কত মাস লাগবে, আর কী লাভ?" সংখ্যা দুইটা পাশাপাশি লিখলেই উত্তর স্পষ্ট হয়

বীমা

আপনি ছাড়া প্রজেক্ট চলবে?

দস্তাবেজ = bus factor-এর একমাত্র প্রতিকার

৫০০+ লাইনের দস্তাবেজ

START_HERE ২১১ · USER_MANUAL ২০৩ · BUILD_STATUS ১৭০ · LAUNCH_PLAYBOOK ১৩৬ · DEPLOYMENT ১২২ · README ৫৮

দস্তাবেজও কোড

`docs/gen_feature_catalogue.py` · `gen_proof_pdf.py` · `gen_roadmap.py` · `build_status.py` · `shoot.py` (৫৩ স্ক্রিনশট)

কেন স্ক্রিপ্ট

হাতে লেখা দস্তাবেজ পুরোনো হয়; স্ক্রিপ্ট চালালেই ছবি ও সংখ্যা নতুন — BUILD_STATUS-এর "৯০ টেস্ট" এভাবেই বাসি হয়েছে

তবু ফাঁক

স্ক্রিপ্ট চালানোর কথা কেউ মনে রাখে না — রিলিজ চেকলিস্টে "দস্তাবেজ পুনরুত্পাদন" ধাপ যোগ করুন

```
$ python docs/build_status.py
$ python docs/gen_feature_catalogue.py
$ python docs/gen_proof_pdf.py
$ python docs/gen_roadmap.py
$ python docs/shoot.py     # ৫৩ স্ক্রিনশট

# এক কমান্ডে সব (এখনো নেই — বানাতে হবে)
$ bash docs/rebuild_docs.sh
```

শিক্ষকের নোট: শেষ লাইনটি সত্যি নয় — এমন `rebuild_docs.sh` ফাইল এখনো নেই; শিক্ষার্থীদের বলুন সেটি বানানোই প্রথম কাজ (সৎ উদাহরণ: দস্তাবেজও দাবি করে, যাচাই করতে হয়)

অনুশীলন ও আলোচনা

২০ মিনিট — এক বছরের পরিকল্পনা লিখুন

চারটি ছোট ফলাফল

১. একটি ADR
নিজের প্রজেক্টের একটি সিদ্ধান্ত — প্রসঙ্গ, সিদ্ধান্ত, বাদ পড়া বিকল্প, পরিণতি

২. ৭০/২০/১০ বাজেট
পরের তিন মাসের কাজ তিন ভাগে ভাগ করুন

৩. ডেটা-নীতি
কোন টেবিল কত দিন রাখবেন — পাঁচটি সারি লিখুন

৪. bus factor পরীক্ষা
একজন সহপাঠীকে দিন — সে কি আপনার প্রজেক্ট চালু করতে পারল?

- কেন ADR আলাদা নথি?
- ঋণ ০% রাখলে কী হয়?
- রিওয়াইটের আগে প্রথম প্রশ্ন?
- দস্তাবেজ পুরোনো হলে কীভাবে বুঝবেন?
- কোন ডেটা কখনো মুছবেন না?

শিক্ষকের নোট: চতুর্থ কাজটি সত্যিই করান — এক সহপাঠীকে ২০ মিনিট দিন; যেখানে আটকে যায়, সেটাই দস্তাবেজের ফাঁক

পর্ব ৭ সমাপ্তি

দুই ক্লাসে যা শেখা হলো

৩৪ স্লাইড · রক্ষণাবেক্ষণের পুরো ছবি

ক্লাস	মূল ধারণা	প্রমাণ
৩১ · কোড বড় হয়	মানচিত্র · ঋণ · রিফ্যাক্টর · নির্ভরতা · বৃদ্ধি	৬,৮৩৩ লাইন · ১,৬০,০০০ উপস্থিতি সারি
৩২ · এক বছর পর	ADR · রিলিজ · ফিচার বনাম ঋণ · রিওয়াইট নয়	১৯ কমিট · ৯০০ লাইন দস্তাবেজ · ৫৩ স্ক্রিনশট

মনে রাখুন: কোড বাড়লে **নিয়ম** বাড়াতে হয়, আর নিয়ম লিখে রাখতে হয় — নইলে তা কেবল আপনার মাথায় থাকে

সৎ ফাঁক: ADR ফোল্ডার নেই · git tag/CHANGELOG নেই · দস্তাবেজ পুনরুত্পাদনের এক-কমান্ড স্ক্রিপ্ট নেই

পরের পর্বে (৮): দস্তাবেজ — এই ফাঁকগুলো পূরণ করার ক্লাস ব্যবহারকারীর জন্য লেখা, ডেভেলপারের জন্য লেখা, আর প্রমাণের দস্তাবেজ

শিক্ষকের নোট: পর্ব ৮-এর সূচনা এই ফাঁকগুলো থেকেই — শিক্ষার্থীরা যেন দেখে দস্তাবেজ কেবল সাজসজ্জা নয়, প্রকৌশলের অংশ

৩৩

পর্ব ৮ — দস্তাবেজ

ব্যবহারবিধি · ডেভেলপার গাইড · প্রমাণের দস্তাবেজ · স্ক্রিনশট · বাংলায় লেখা